Problem Editorial Submissions Comments

Reverse an Array

Difficulty: **Easy** Accuracy: **55.32%** Submissions: **266K+** Points: **2** Average Time: **5m**

You are given an array of integers **arr[]**. You have to **reverse** the given array.

Note: Modify the array in place.

Examples:

Input: arr = [1, 4, 3, 2, 6, 5]
Output: [5, 6, 2, 3, 4, 1]
Explanation: The elements of the array are [1, 4, 3, 2, 6, 5]. After reversing the array, the first element goes to the last position, the second

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed

1115 / 1115

Attempts : Correct / Total

1 / 1

Accuracy : 100%

Java (21) Start Timer

```java
class Solution {
    public static void reverseArray(int[] arr) {
        int i = 0, j = arr.length - 1;
        while (i < j) {
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
            i++;
            j--;
        }
    }
}
```

Custom Input Compile & Run Submit

Min and Max in Array

Difficulty: **Basic** Accuracy: **68.55%** Submissions: **402K+** Points: **1** Average Time: **10m**

Given an array **arr[]**. Your task is to find the **minimum** and **maximum** elements in the array.

Examples:

Input: arr[] = [1, 4, 3, 5, 8, 6]
Output: [1, 8]
Explanation: minimum and maximum elements of array are 1 and 8.

Java (21) Start Timer

```java
import java.util.ArrayList;

class Solution {
    public static ArrayList<Integer> getMinMax(int[] arr) {
        int min = arr[0], max = arr[0];

        for (int i = 1; i < arr.length; i++) {
            if (arr[i] < min) min = arr[i];
            if (arr[i] > max) max = arr[i];
        }

        ArrayList<Integer> res = new ArrayList<>();
        res.add(min);
        res.add(max);
        return res;
    }
}
```

Custom Input Compile & Run Submit

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed
1111 / 1111

Attempts : Correct / Total
1 / 1
Accuracy : 100%

Problem Editorial Submissions Comments

Java (21) Start Timer

Kth Smallest

Difficulty: **Medium** Accuracy: **35.17%** Submissions: **736K+** Points: **4**

Average Time: **25m**

Given an integer array **arr[]** and an integer **k**, your task is to find and return the **k**th **smallest** element in the given array.

Note: The kth smallest element is determined based on the sorted order of the array.

Examples :

Input: arr[] = [10, 5, 4, 3, 48, 6, 2, 33, 53, 10], k = 4
Output: 5

```java
import java.util.Arrays;

class Solution {
    public static int kthSmallest(int[] arr, int k) {
        Arrays.sort(arr);
        return arr[k - 1];
    }
}
```

Custom Input Compile & Run Submit

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed

1121 / 1121

Attempts : Correct / Total

1 / 1

Accuracy : 100%

Union of Arrays with Duplicates

Difficulty: **Easy** Accuracy: **42.22%** Submissions: **480K+** Points: **2** Average Time: **10m**

You are given two arrays **a[]** and **b[]**, return the **Union** of both the arrays in any order.

The **Union** of two arrays is a collection of all **distinct elements** present in either of the arrays. If an element appears more than once in one or both arrays, it should be included **only once** in the result.

Note: Elements of **a[]** and **b[]** are not necessarily distinct.
Note that, You can return the Union in any order but the driver code will print the result in **sorted order** only.

Examples:

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed

1111 / 1111

Attempts : Correct / Total

1 / 1

Accuracy : 100%

Java (21) Start Timer

```java
import java.util.ArrayList;
import java.util.HashSet;

class Solution {
    public static ArrayList<Integer> findUnion(int[] a, int[] b) {
        HashSet<Integer> set = new HashSet<>();

        for (int x : a) set.add(x);
        for (int x : b) set.add(x);

        return new ArrayList<>(set);
    }
}
```

Custom Input Compile & Run Submit

Largest in Array

Difficulty: **Basic** Accuracy: **67.48%** Submissions: **562K+** Points: **1** Average Time: **20m**

Given an array **arr[].** The task is to find the largest element and return it.

Examples:

Input: arr[] = [1, 8, 7, 56, 90]
Output: 90
Explanation: The largest element of the given array is 90.

Input: arr[] = [5, 5, 5, 5]

Java (21) Start Timer

```java
class Solution {
    public static int largest(int[] arr) {
        int max = arr[0];
        for (int i = 1; i < arr.length; i++) {
            if (arr[i] > max) max = arr[i];
        }
        return max;
    }
}
```

Custom Input Compile & Run Submit

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed
1115 / 1115

Attempts : Correct / Total
1 / 1
Accuracy : 100%

Rotate Array by One

Difficulty: **Basic** Accuracy: **69.6%** Submissions: **353K+** Points: **1** Average Time: **20m**

Given an array **arr**, rotate the array by one position in clockwise direction.

Examples:

Input: arr[] = [1, 2, 3, 4, 5]
Output: [5, 1, 2, 3, 4]
Explanation: If we rotate arr by one position in clockwise 5 come to the front and remaining those are shifted to the end.

Java (21) Start Timer

```java
class Solution {
    public static void rotate(int[] arr) {
        int n = arr.length;
        if (n <= 1) return;

        int last = arr[n - 1];
        for (int i = n - 1; i > 0; i--) {
            arr[i] = arr[i - 1];
        }
        arr[0] = last;
    }
}
```

Custom Input Compile & Run Submit

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed
1115 / 1115

Attempts : Correct / Total
1 / 1
Accuracy : 100%

Kadane's Algorithm

Difficulty: **Medium** Accuracy: **36.28%** Submissions: **1.2M** Points: **4** Average Time: **20m**

You are given an integer array **arr[].** You need to find the **maximum** sum of a subarray (containing at least one element) in the array **arr[]**.

Note : A **subarray** is a continuous part of an array.

Examples:

Input: arr[] = [2, 3, -8, 7, -1, 2, 3]
Output: 11
Explanation: The subarray [7, -1, 2, 3] has the largest sum 11.

Java (21) Start Timer

```java
class Solution {
    public static int maxSubarraySum(int[] arr) {
        int maxSoFar = arr[0];
        int currMax = arr[0];

        for (int i = 1; i < arr.length; i++) {
            currMax = Math.max(arr[i], currMax + arr[i]);
            maxSoFar = Math.max(maxSoFar, currMax);
        }
        return maxSoFar;
    }
}
```

Custom Input Compile & Run Submit

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed
1120 / 1120

Attempts : Correct / Total
1 / 1
Accuracy : 100%

35. Search Insert Position

Easy | Topics | Companies

Given a sorted array of distinct integers and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order.

You must write an algorithm with `O(log n)` runtime complexity.

Example 1:

```
Input: nums = [1,3,5,6], target = 5
Output: 2
```

Example 2:

```
Input: nums = [1,3,5,6], target = 2
Output: 1
```

Example 3:

```
Input: nums = [1,3,5,6], target = 7
```

18.5K 413 350 Online

Code

Java Auto

```java
class Solution {
    public int searchInsert(int[] nums, int target) {
        int left = 0, right = nums.length - 1;

        while (left <= right) {
            int mid = left + (right - left) / 2;

            if (nums[mid] == target)
                return mid;
```

Saved Ln 18, Col 1

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

nums =
[1,3,5,6]

target =

Description | Editorial | Solutions | Submissions

1. Two Sum

Easy | Topics | Companies | Hint

Given an array of integers `nums` and an integer `target`, return *indices of the two numbers such that they add up to* `target`.

You may assume that each input would have ***exactly* one solution**, and you may not use the *same* element twice.

You can return the answer in any order.

Example 1:

```
Input: nums = [2,7,11,15], target = 9
Output: [0,1]
Explanation: Because nums[0] + nums[1] == 9, we return [0, 1].
```

Example 2:

```
Input: nums = [3,2,4], target = 6
Output: [1,2]
```

67K 1.8K
3391 Online

Code

Java | Auto

```
                return new int[]{map.get(need), i};
            }
            map.put(nums[i], i);
        }
        return new int[]{}; // will never reach as one solution always exists
    }
}
```

Saved
Ln 17, Col 1

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

nums =
[2,7,11,15]

target =

Minimum Jumps

Difficulty: **Medium** Accuracy: **11.91%** Submissions: **1.1M** Points: **4**

You are given an array **arr[]** of non-negative numbers. Each number tells you the **maximum number of steps** you can jump forward from that position.

For example:

- If **arr[i] = 3**, you can jump to index **i + 1**, **i + 2**, or **i + 3** from position **i**.
- If **arr[i] = 0**, you **cannot jump forward** from that position.

Your task is to find the **minimum number of jumps** needed to move from the **first** position in the array to the **last** position.

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed

1120 / 1120

Attempts : Correct / Total

1 / 1

Accuracy : 100%

Java (21) Start Timer

```java
class Solution {
    public static int minJumps(int[] arr) {
        int n = arr.length;
        if (n <= 1) return 0;
        if (arr[0] == 0) return -1;

        int jumps = 1;
        int maxReach = arr[0];
        int steps = arr[0];

        for (int i = 1; i < n; i++) {
            if (i == n - 1) return jumps;

            maxReach = Math.max(maxReach, i + arr[i]);
            steps--;

            if (steps == 0) {
                jumps++;
                if (i >= maxReach) return -1;
                steps = maxReach - i;
            }
        }
        return -1;
    }
}
```

Custom Input Compile & Run Submit